

Multithreading Practice – CO3

1.



SIMATS Online Programming Lab
JAVA PROGRAMMING

K.Othup
19230541390

← Back

QUESTIONS

Q1
Multiple Threads Using a Single Class
10 marks

Q2
Multiple Thread classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

Q1: Multiple Threads Using a Single Class

10 marks

Topic: Thread class, multiple threads, if-else

Question:
Write a Java program using a single thread class to create three threads named Prime, Armstrong, and Reverse.

The program should contain a common run() method. Based on the thread name, use if-else statements to perform the following operations on a given number:

Prime → Check whether the number is prime.
Armstrong → Check whether the number is an Armstrong number.
Reverse → Find the reverse of the number.

Create and start all three threads from the main() method.

Input: 153

Expected Output:

Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 = 351

Your Code

```
1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[] args) throws InterruptedException{
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         NumberThread t1 = new NumberThread("Prime", n);
7         NumberThread t2 = new NumberThread("Armstrong", n);
8         NumberThread t3 = new NumberThread("Reverse", n);
9         t1.start();
10        t1.join();
11        t2.start();
12        t2.join();
13        t3.start();
14        t3.join();
15    }
16 }
17 class NumberThread extends Thread{
18     int n;
19     NumberThread(String name, int n){
20         super(name);
21         this.n = n;
22     }
23     public void run() {
24         if (getName().equals("Prime")){
25             boolean prime = true;
26             if (n < 2)
27                 prime = false;
28             for (int i = 2; i <= n; i++){
29                 if (n % i == 0) {
30                     prime = false;
31                 }
32             }
33         }
34     }
35 }
```

Input

153

Output

Prime Thread: 153 is not a prime number
Armstrong Thread: 153 is an Armstrong number
Reverse Thread: Reverse of 153 = 351

Visible Test Cases

Test Case 1 ✓
Test Case 2 ✓
Test Case 3 ✓

10.00 / 10 marks

2.

SIMATS Online Programming Lab
 JAVA PROGRAMMING

K.Dhillip
 19236503890

Multithreading Practice - CO3

QUESTIONS

Q1
Multiple Threads Using a Single Class
10 marks

Q2
Multiple Thread Classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

Q2 Multiple thread classes

20 marks

Question:
Write a Java program to create three different thread classes to perform different operations on the same number.

Create the following classes:

PrimeThread → Check whether the given number is prime.
 FactorialThread → Find the factorial of the given number.
 ReverseThread → Find the reverse of the given number.

Each class should extend the Thread class and override the run() method.
 Create and start all three threads from the main() method.

Input: 7

Expected Output:

Prime: 7 is a prime number
 Factorial: 7! = 5040
 Reverse: Reverse of 7 = 7

Note: Since the threads execute concurrently, the order of output may vary.

Your Code

```

1 import java.util.Scanner;
2 public class Main{
3     public static void main(String[] args) throws Exception{
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         PrimeThread t1 = new PrimeThread(n);
7         FactorialThread t2 = new FactorialThread(n);
8         ReverseThread t3 = new ReverseThread(n);
9         t1.start();
10        t1.join();
11        t2.start();
12        t2.join();
13        t3.start();
14        t3.join();
15    }
16 }
17 class PrimeThread extends Thread{
18     int n;
19     PrimeThread(int n){
20         this.n=n;
21     }
22     public void run(){
23         boolean prime=true;
24         if (n < 2)
25             prime=false;
26         for (int i=2;i<=n;i++){
27             if (n % i == 0){
28                 prime = false;
29             }

```

Input

7

Output

```

Prime: 7 is a prime number
Factorial: 7! = 5040
Reverse: Reverse of 7 = 7

```

Visible Test Cases

Test Case 1

Test Case 2

Test Case 3

20.00 / 20 marks

3.



SIMATS Online Programming Lab
JAVA PROGRAMMING

 **K.Dhilip**
16018AN01982

Multithreading Practice - CO3
Back

QUESTIONS

Q1
Multiple Threads Using a Single Class
10 marks

Q2
Multiple Thread classes
20 marks

Q3
Thread Synchronization
10 marks

3/3 answered

Q3 Thread Synchronization
10 marks

topic: synchronized, shared resource, race condition

Question:

Write a Java program to demonstrate thread synchronization using a shared Counter object.

Create a class Counter containing an integer variable count, initially set to 0.

Create two threads, ThreadA and ThreadB. Both threads should increment the same count variable 1000 times.

Implement the increment() method using the synchronized keyword so that only one thread can modify the counter at a time.

After both threads complete their execution, display the final value of count.

Expected Output:

```
ThreadA completed
ThreadB completed
Final Count = 2000
```

Requirements:

- Use a shared Counter object.
- Both threads must access the same Counter object.
- Use synchronized for the increment operation.
- Use join() to ensure both threads finish before displaying the final count.

Your Code

```

1 import java.util.*;
2 public class Main{
3     public static void main(String[] args) throws InterruptedException{
4         Counter c=new Counter();
5         ThreadA t1=new ThreadA(c);
6         ThreadB t2=new ThreadB(c);
7         t1.start();
8         t2.start();
9         t1.join();
10        t2.join();
11        System.out.println("Final Count = "+c.count);
12    }
13 }
14 class Counter{
15     int count=0;
16     synchronized void increment(){
17         count++;
18     }
19 }
20 class ThreadA extends Thread{
21     Counter c;
22     ThreadA(Counter c){
23         this.c=c;
24     }
25     public void run(){
26         for (int i=0;i<1000;i++){

```

Input

stdin input

Output

```

ThreadA completed
ThreadB completed
Final Count = 2000

```

Visible Test Cases

Test Case 1 ✓

Test Case 2 ✓

Test Case 3 ✓

10.00 / 10 marks